

=====

PROJECT: MULTI-AGENT AUTOMATED COMPLIANCE VERIFICATION PIPELINE

=====

PREPARED BY:
Smruti Rout
Senior Advanced Software Architect

DATE OF AUDIT:
June 26, 2026

TARGET ECOSYSTEMS:
Python • Java • C++ (CMake)

FOR DEVELOPMENT USE

PROJECT DOCUMENTATION

MULTI-AGENT AUTOMATED COMPLIANCE VERIFICATION PIPELINE

REAL-TIME SBOM SYNTHESIS AND DETERMINISTIC NVD RISK
MITIGATION FOR EU CYBER RESILIENCE ACT COMPLIANCE



FOR DEVELOPMENT USE

CRA

Project Report: Multi-Agent Automated Compliance Verification Pipeline

Subtitle: Implementing Real-Time SBOM Synthesis and Deterministic NVD Risk Mitigation for EU Cyber Resilience Act Compliance

1. Executive Summary

The implementation of the European Union Cyber Resilience Act (CRA) introduces strict statutory mandates for digital products shipped to the European market. Specifically, Article 10 rules ban the commercial introduction of software components containing known, actively exploitable vulnerabilities.

To address this, this project delivers a **Multi-Agent Automated Compliance Verification Pipeline** powered by LangGraph, Pydantic V2, and Streamlit. The pipeline automates the generation of a Software Bill of Materials (SBOM) across polyglot ecosystems (Python, Java, C++) and maps targets to the National Vulnerability Database (NVD) via strict Common Platform Enumeration (CPE) matching. It features an intelligent text-filtering engine, automated patch advisories, and an audit-ready compliance override framework.

2. System Architecture & Modular Design Overview

The system relies on a strictly decoupled architecture, isolating parsing mechanics, API connection engines, state orchestration, and user interface delivery into dedicated modules.

Component Map

- **schemas.py**: State configuration layer utilizing Pydantic V2 models to guarantee mathematical runtime telemetry safety across execution boundaries.
 - **pipeline.py**: Graph orchestration framework powered by LangGraph, routing pipeline updates securely through independent, state-bound node operations.
 - **agents.py**: Functional computation core containing multi-agent parsing regex scripts, NVD API pool managers, and whitelist validator frameworks.
 - **app.py**: Web-based presentation layer built with Streamlit, handling concurrent multi-manifest file streaming buffers seamlessly.
-

3. Engineering Analysis: Solving Core Pipeline Failure Vectors

3.1 Resolving OpenSSL Non-Standard Version Profiling

Legacy software components frequently utilize trailing alphabet indicators (e.g., OpenSSL `1.0.1f`), which instantly break conventional PEP-440 semantic version parsers. This pipeline implements a mathematical normalization wrapper:

$$f(c) = \text{ord}(c) - \text{ord}('a') + 1$$

This function safely normalizes non-standard version inputs (such as `1.0.1f` into `1.0.1.6`). This keeps boundary validation blocks from crashing and ensures historical security risks like the **Heartbleed** exploit vector are correctly identified.

3.2 Transitioning from Keyword Search to Deterministic CPE Name Matching

Relying on generic API keyword lookups introduces significant noise, flooding results with unrelated description items and hitting rate limits. To solve this, this pipeline maps dependencies straight to their official internal NVD product footprints via an automated translation lookup dictionary:

Source Identifier	Ecosystem	NVD Mapped Vendor	NVD Mapped Product
django	Python / PIP	django	django
requests	Python / PIP	python	requests
log4j-core	Java / Maven	apache	log4j
openssl	C++ / CMake	openssl	openssl

By mapping components straight to strict formatting matrices (`cpe:2.3:a:{vendor}:{product}:{version}`), lookups execute cleanly with pinpoint precision.

3.3 Network Defenses Against Slow NVD Infrastructures

The NVD API frequently drops connections and throttles users. This system hardens the network connection layer inside `agents.py` using three strategies:

1. **Extended Timeout Thresholds:** Connection limits were bumped from 30 to 60 seconds to absorb slow response times.
 2. **Backoff Retries:** Integrated an `HTTPAdapter` retry loop that automatically catches unexpected network drops, pacing retries dynamically over progressive delays (2s, 4s, 8s, 16s).
 3. **Throttling Guardrails:** Injected a 5-second sleep delay into unauthenticated pipeline loops to stay safely under rate-limiting windows.
-

4. Cyber Resilience Act Audit & Exclusions Engine

4.1 Strict Article 10 Policy Enforcement

The system's final evaluation node functions as a automated compliance inspector. It enforces an absolute zero-trust policy model:

```
$$\text{CRA Compliant} = \begin{cases} \text{True} & \text{if } \sum \text{Exploitable Vulns} = 0 \\ \text{AND } \sum \text{SBOM Items} > 0 \\ \text{False} & \text{otherwise} \end{cases}$$
```

If even one unmitigated threat with a CVSS score indicating a high or critical risk runs through the pipeline, the status swaps to a hard **FAIL**, blocking production distribution to the EU.

4.2 Automated Patch Advisory Layer

When a vulnerability is detected, the system extracts boundary properties like `versionEndExcluding` or `versionEndIncluding` directly from the NVD data node. This lets the pipeline provide clear remediation advice right in the UI, telling developers exactly which safe version to upgrade to.

4.3 Auditable Whitelist Justification

To handle edge cases where a vulnerability cannot be exploited in production, developers can register exceptions in a structured config file (`cra-whitelist.yaml`). To comply with CRA audit guidelines, every entry must include a clear technical justification:

```
whitelisted_cves:  
- cve_id: "CVE-2021-44228"  
  reason: "Java runtime engine is hardcoded with system property config flag  
'-Dlog4j2.formatMsgNoLookups=true', permanently disabling the JNDI vector."
```

The pipeline parses these overrides using explicit **UTF-8 character encoding**, ensuring special characters and formatting text load smoothly without breaking cross-platform builds.

5. UI Integration & Adaptive Presentation Layer

The presentation frontend (`app.py`) wraps these deep compliance checks in a clean, professional web browser interface.

UI Architecture Features

- **File Upload Buffers:** Built distinct file upload widgets for `requirements.txt`, `pom.xml`, and `CMakeLists.txt`, streaming inputs to isolated temporary folders to keep the main directory clean.
 - **Adaptive Theme Support:** Replaced hardcoded CSS background colors with native `st.container(border=True)` hooks, ensuring excellent text contrast in both light and dark mode.
 - **Interactive Expandable Drawers:** Moved heavy text blocks out of the data tables and into interactive dropdown panels, keeping the main dashboard clean and organized.
 - **Highlighted Vulnerability Feeds:** Uses regular expression filters to auto-bold and color-code critical alert phrases like `RCE` or `Remote Code Execution` in bright red, making severe threats stand out instantly during an audit.
-

6. Conclusion & Roadmap

This multi-agent verification pipeline successfully mitigates the primary automation hurdles of the Cyber Resilience Act. By combining precise CPE matching with flexible fuzzy exceptions, it gives development teams a fast, secure, and clear compliance checkpoint.

Future upgrades can expand the pipeline by adding automated **CycloneDX report exports** and integrating alternative security data feeds like **OSV.dev** to provide secondary network backups.

Testing Artifacts:

EU EU Cyber Resilience Act (CRA) Pipi X

localhost:8501

Deploy

EU Cyber Resilience Act (CRA) Audit Pipeline

Live Multi-Agent Compliance Scan & Direct Manifest File Upload Platform

Upload Target Manifest Files

Upload one or more manifest files below to evaluate project dependencies live against NVD records.

Python Environment

requirements.txt232.0B

+

Java Project Object Model

pom.xml0.5KB

+

CMake Build System Configuration

CMakeLists.txt299.0B

+

Run Compliance Audit

✗ FAIL: CRITICAL NON-COMPLIANCE DETECTED

Known exploitable vulnerabilities exist. This software cannot be legally shipped to the EU market.

Generated Software Bill of Materials (SBOM)

Component	Version	Language	Original Manifest
django	2.2.0	Python	requirements.txt
requests	2.28.1	Python	requirements.txt

NVD Live Vulnerability Analysis

CVE ID	Target Package	Version	CVSS Score	Severity	Status
CVE-2019-19844	django	2.2.0	9.8	CRITICAL	⚠️ AUDIT TRACKED
CVE-2020-7471	django	2.2.0	9.8	CRITICAL	⚠️ AUDIT TRACKED

✗ FAIL: CRITICAL NON-COMPLIANCE DETECTED

Known exploitable vulnerabilities exist. This software cannot be legally shipped to the EU market.

Generated Software Bill of Materials (SBOM)

Component	Version	Language	Original Manifest
django	2.2.0	Python	requirements.txt
requests	2.28.1	Python	requirements.txt
fastapi	0.100.0	Python	requirements.txt
log4j-core	2.14.1	Java	pom.xml
json	20230227	Java	pom.xml
openssl	1.0.1f	C++	CMakeLists.txt
threads	REQUIRED	C++	CMakeLists.txt

NVD Live Vulnerability Analysis

CVE ID	Target Package	Version	CVSS Score	Severity	Status
CVE-2021-44228	log4j-core	2.14.1	10	CRITICAL	✅ ALLOWED (WHITELISTED)
CVE-2019-14234	django	2.2.0	9.8	CRITICAL	✅ ALLOWED (WHITELISTED)
CVE-2019-19844	django	2.2.0	9.8	CRITICAL	⚠️ AUDIT TRACKED
CVE-2020-7471	django	2.2.0	9.8	CRITICAL	⚠️ AUDIT TRACKED
CVE-2022-28346	django	2.2.0	9.8	CRITICAL	⚠️ AUDIT TRACKED
CVE-2022-28347	django	2.2.0	9.8	CRITICAL	⚠️ AUDIT TRACKED
CVE-2021-45046	log4j-core	2.14.1	9	CRITICAL	🚫 BLOCKS SHIPMENT
CVE-2020-9402	django	2.2.0	8.8	HIGH	⚠️ AUDIT TRACKED
CVE-2019-14232	django	2.2.0	7.5	HIGH	⚠️ AUDIT TRACKED
CVE-2019-14233	django	2.2.0	7.5	HIGH	⚠️ AUDIT TRACKED

🔍 Interactive Vulnerability Breakdown Feed

Interactive Vulnerability Breakdown Feed

Click on any threat card below to read the full NVD description and see the remediation roadmap.

▼  CVE-2019-12308 — DJANGO (2.2.0) | CVSS: 6.1 (MEDIUM)

CRA Regulatory Impact:  AUDIT TRACKED (LOW/MEDIUM COMPLIANCE RISK)

 **CRA Remediation Patch Advisory:** Upgrade to >= 2.2.2

An issue was discovered in Django 1.11 before 1.11.21, 2.1 before 2.1.9, and 2.2 before 2.2.2. The clickable Current URL value displayed by the AdminURLFieldWidget displays the provided value without validating it as a safe URL. Thus, an unvalidated value stored in the database, or a value provided as a URL query parameter payload, could result in an clickable JavaScript link.

▼  CVE-2019-12781 — DJANGO (2.2.0) | CVSS: 5.3 (MEDIUM)

CRA Regulatory Impact:  AUDIT TRACKED (LOW/MEDIUM COMPLIANCE RISK)

 **CRA Remediation Patch Advisory:** Upgrade to >= 2.2.3

An issue was discovered in Django 1.11 before 1.11.22, 2.1 before 2.1.10, and 2.2 before 2.2.3. An HTTP request is not redirected to HTTPS when the SECURE_PROXY_SSL_HEADER and SECURE_SSL_REDIRECT settings are used, and the proxy connects to Django via HTTPS. In other words, django.http.HttpRequest.scheme has incorrect behavior when a client uses HTTP.

▶  CVE-2019-14232 — DJANGO (2.2.0) | CVSS: 7.5 (HIGH)

 **CRA Remediation Patch Advisory:** Upgrade to >= 2.2.25

In Django 2.2 before 2.2.25, 3.1 before 3.1.14, and 3.2 before 3.2.10, HTTP requests for URLs with trailing newlines could **bypass** upstream access control based on URL paths.

▼  CVE-2021-45115 — DJANGO (2.2.0) | CVSS: 7.5 (HIGH)

CRA Regulatory Impact:  AUDIT TRACKED (LOW/MEDIUM COMPLIANCE RISK)

 **CRA Remediation Patch Advisory:** Upgrade to >= 2.2.26

An issue was discovered in Django 2.2 before 2.2.26, 3.2 before 3.2.11, and 4.0 before 4.0.1. UserAttributeSimilarityValidator incurred significant overhead in evaluating a submitted password that was artificially large in relation to the comparison values. In a situation where access to user registration was unrestricted, this provided a potential vector for a denial-of-service attack.

▼  CVE-2021-45116 — DJANGO (2.2.0) | CVSS: 7.5 (HIGH)

CRA Regulatory Impact:  AUDIT TRACKED (LOW/MEDIUM COMPLIANCE RISK)

 **CRA Remediation Patch Advisory:** Upgrade to >= 2.2.26

An issue was discovered in Django 2.2 before 2.2.26, 3.2 before 3.2.11, and 4.0 before 4.0.1. Due to leveraging the Django Template Language's variable resolution logic, the dictsort template filter was potentially vulnerable to **information disclosure**, or an unintended method call, if passed a suitably crafted key.

 CRA Remediation Patch Advisory: [Check](#) [Vendor](#) [Advisory](#)

Memory **leak** in the `tls_decrypt_ticket` function in `tl1_lib.c` in OpenSSL before 0.9.8zc, 1.0.0 before 1.0.0o, and 1.0.1 before 1.0.1j allows remote attackers to cause a **denial of service** (memory consumption) via a crafted session ticket that triggers an integrity-check failure.

✓  CVE-2014-3568 — OPENSSL (1.0.1f) | CVSS: 4.3 (MEDIUM)

CRA Regulatory Impact:  BLOCKS EU SHIPMENT (HIGH/CRITICAL EXPLOIT RISK)

 CRA Remediation Patch Advisory: [Check](#) [Vendor](#) [Advisory](#)

OpenSSL before 0.9.8zc, 1.0.0 before 1.0.0o, and 1.0.1 before 1.0.1j does not properly enforce the `no-ssl3` build option, which allows remote attackers to **bypass** intended access restrictions via an SSL 3.0 handshake, related to `s23_clnt.c` and `s23_srvr.c`.

✓  CVE-2014-3570 — OPENSSL (1.0.1f) | CVSS: 5.0 (MEDIUM)

CRA Regulatory Impact:  AUDIT TRACKED (LOW/MEDIUM COMPLIANCE RISK)

 CRA Remediation Patch Advisory: [Check](#) [Vendor](#) [Advisory](#)

The `BN_sqr` implementation in OpenSSL before 0.9.8zd, 1.0.0 before 1.0.0p, and 1.0.1 before 1.0.1k does not properly calculate the square of a BIGNUM value, which might make it easier for remote attackers to defeat cryptographic protection mechanisms via unspecified vectors, related to `crypto/bn/asm/mips.pl`, `crypto/bn/asm/x86_64-gcc.c`, and `crypto/bn/bn_asm.c`.